

Spring Boot Caching with Redis

Complete Guide to Caching Strategies in Microservices

Table of Contents

1. [Introduction to Caching](#)
 2. [Understanding Cache Layers](#)
 3. [Local vs Distributed Caching](#)
 4. [Cache Technologies Compared](#)
 5. [Caching Patterns](#)
 6. [Spring Cache Abstraction](#)
 7. [Implementing @Cacheable](#)
 8. [Cache Invalidation Strategies](#)
 9. [Advanced Redis Configuration](#)
 10. [Live Coding Workshop](#)
 11. [Common Pitfalls and Solutions](#)
 12. [Quick Reference](#)
-

1. Introduction to Caching

What is Caching?

A cache is a temporary storage that keeps frequently accessed data in fast memory. Instead of computing or loading data repeatedly, a cache stores the result so it can be retrieved quickly.

Key Insight: Caching trades memory for speed. Instead of spending CPU and database resources repeatedly, we spend a small amount of RAM.

Why Do We Need Caching?

Consider this scenario:

- Your Product Service receives 100,000 requests per hour
- Each request queries the database for the same products
- The database does unnecessary work answering the same queries repeatedly

Without caching:

```
Client → Product Service → PostgreSQL (same query, 100,000 times)
```

With caching:

```
Client → Product Service → Redis Cache → (cache hit, no database query)
```

Access Time Comparison

Storage	Approximate Latency
CPU Cache	< 1 ns
RAM	50–100 ns
SSD	50–150 μ s
HDD	5–100 ms
Network	50–1000 ms

Remember: The farther data is from the CPU, the slower it becomes. Caching brings data closer.

Important Distinction

Caching is a design technique, not a technology.
Redis is a technology that implements caching.
Spring Cache is an abstraction over caching technologies.

2. Understanding Cache Layers

Modern systems already use many caches simultaneously. Let's understand each layer:

Layer 1 - CPU Cache

- **What:** Tiny memory inside the processor
- **Purpose:** Reduce memory access latency
- **Who manages:** Operating system and hardware (automatic)

Layer 2 - Application Cache

- **What:** Cache we create in our application code
- **Examples:** HashMap, Caffeine, EhCache
- **Purpose:** Store objects in memory instead of querying databases

Layer 3 - Hibernate First-Level Cache (L1)

- **What:** Built-in cache in every EntityManager
- **Scope:** Single transaction only
- **Auto-enabled:** Yes, no configuration needed

Example:

```
// Inside one transaction
Product p1 = entityManager.find(Product.class, 1L); // Database query
Product p2 = entityManager.find(Product.class, 1L); // From memory - no query!
```

Layer 4 - Hibernate Second-Level Cache (L2)

- **What:** Cache that survives beyond one transaction
- **Scope:** Multiple EntityManagers in same application
- **Popular providers:** EhCache, Infinispan, Hazelcast

Why less common in Microservices:

- Multiple pods would have inconsistent caches
- Synchronization becomes difficult

Layer 5 - Spring Cache

- **What:** Abstraction layer for caching

- **Purpose:** Provides a common API (@Cacheable, @CacheEvict)
- **Not:** An actual cache implementation

Think of it like:

```
Spring Cache = SLF4J (abstraction)
Redis = Logback (implementation)
```

3. Local vs Distributed Caching

Local Cache

Definition: Cache stored inside the application's own memory (JVM Heap).

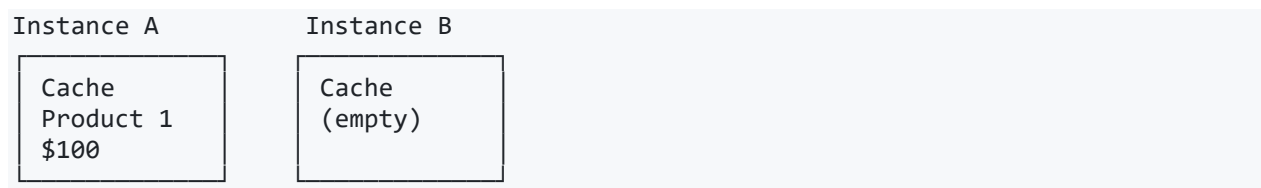
Advantages

- Extremely fast
- No network call
- Very low latency
- Very simple
- Easy to configure

Disadvantages

- Cache disappears if application restarts
- Every application instance has its own copy
- Difficult to keep multiple instances synchronized

Visual Example



Problem: If requests go to Instance B, it must query the database again.

Distributed Cache

Definition: Cache stored in a dedicated cache server shared by all instances.

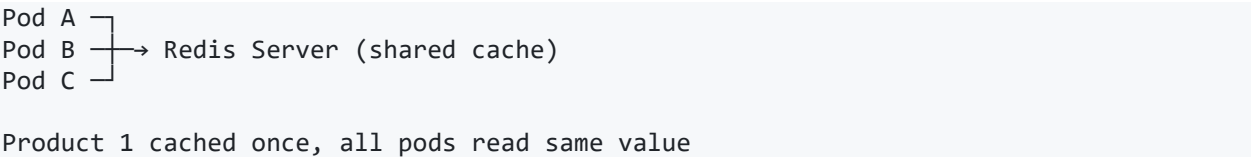
Advantages

- Shared by all instances
- Consistent across pods
- Works perfectly with Kubernetes
- Easy to scale horizontally
- Independent of application lifecycle

Disadvantages

- Small network latency
- Extra infrastructure
- Requires monitoring

Visual Example



Comparison Table

Feature	Local Cache	Distributed Cache
Storage	JVM Memory	External Server
Network Call	No	Yes
Speed	Extremely Fast	Very Fast
Shared	No	Yes
Kubernetes Ready	No	Yes

4. Cache Technologies Compared

HashMap - The Simplest Cache

```
Map<Long, Product> cache = new HashMap<>();
```

Advantages

- Simple
- No dependency
- Fast

Problems

- No expiration
- No TTL (Time To Live)
- No statistics
- No eviction policy
- Unlimited memory growth

When to use: Learning, demos, interview examples. **Never in production.**

Caffeine - Modern Local Cache

Caffeine is a professional HashMap that automatically provides:

- TTL (Time To Live)
- Maximum size limits
- Eviction policies
- Thread safety
- Statistics
- Excellent performance

Example Configuration

```
@Bean  
CacheManager cacheManager() {  
    CaffeineCacheManager manager = new CaffeineCacheManager();
```

```
manager.setCaffeine(  
    Caffeine.newBuilder()  
        .expireAfterWrite(Duration.ofMinutes(5))  
        .maximumSize(1000)  
);  
return manager;  
}
```

Best Use Cases

- Single Spring Boot application
- Monoliths
- Desktop applications
- Batch processing
- Single API instance

Not ideal for: Microservices, Kubernetes, multiple replicas

EhCache - Legacy Enterprise Cache

One of the oldest Java cache libraries. Still common in legacy applications.

Features

- TTL
- Disk persistence
- Memory limits
- Statistics
- Thread safety

Best Use Cases

- Large legacy Java applications
- Older Spring systems
- Java EE applications
- Monoliths

Hazelcast - Distributed Data Grid

Key Difference: Hazelcast is not "another cache" - it's a distributed in-memory data grid.

How It Works

App A ↔ App B ↔ App C
Each node stores part of the data
The cluster behaves like one huge distributed memory

Features

- Distributed Maps
- Distributed Queues
- Distributed Locks
- Distributed Executors
- Distributed Sets
- Distributed Topics
- Near Cache
- Distributed Computing

Best Use Cases

- Distributed computing
- Large in-memory data grids
- Compute clusters
- Complex enterprise platforms

Redis - The Modern Standard

Redis is the most widely used cache for modern microservices.

Key Features

- Strings, Hashes, Lists, Sets, Sorted Sets
- Bitmaps, HyperLogLog
- Pub/Sub, Streams
- Transactions, Lua Scripts
- Distributed Locks

- Replication
- Clustering
- Persistence

Why Redis is described as: "An in-memory data platform" rather than just a cache.

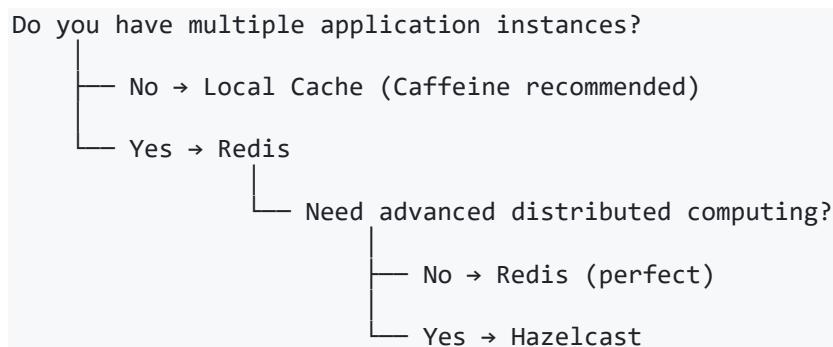
Why Redis Became the Standard

- Shared across all service instances
- No duplicated database queries
- No inconsistent caches
- No synchronization problems
- Scales naturally with more service instances

Choosing the Right Cache

Situation	Recommended Technology
Learning caching	HashMap
Simple local application	Caffeine
Legacy enterprise monolith	EhCache
Single Spring Boot application	Caffeine
Multiple Spring Boot instances	Redis
Kubernetes	Redis
Spring Cloud Microservices	Redis

Decision Tree



5. Caching Patterns

A cache is only a storage mechanism. The more important question is: **How does an application interact with that cache?**

Pattern 1 - Cache-Aside (Lazy Loading)

The most popular caching strategy.

The application is responsible for:

1. Reading from the cache
2. Updating the cache

The cache itself is passive.

Step-by-Step Example

First Request:

1. GET /products/1
2. Cache lookup: Redis → NOT FOUND
3. Database query: `SELECT * FROM products WHERE id = 1`
4. Store result in Redis: `products::1`
5. Return result

Second Request:

1. GET /products/1
2. Cache lookup: Redis → FOUND
3. Return cached result
4. Database is never contacted

Advantages

- Very simple
- Excellent read performance
- Only caches data that is actually used
- Easy to understand

- Supported directly by Spring Cache

Disadvantages

- First request is slower
- Application must remember to invalidate cache
- Cache consistency is the application's responsibility

Why "Lazy Loading"?

Data is not cached until someone requests it. No request → No cache.

Pattern 2 - Read-Through

Application never queries the database directly. It asks the cache, which loads missing data.

```
Application → Cache → Database
               ↑
           If missing, cache loads it
```

Why Not Used in Spring Boot?

Redis doesn't know your repositories, entities, or business logic. Spring prefers Cache-Aside because the application controls loading.

Pattern 3 - Write-Through

Every write goes to cache first, then immediately to database.

```
Application → Cache → Database
                (immediate)
```

Advantages

- Cache always contains latest data

Disadvantages

- Every write becomes slower
- Write latency increases

Common Usage

- Financial systems
- Systems requiring consistent reads

Pattern 4 - Write-Behind (Write-Back)

Application writes only to cache. Database updates happen later in background.

Application → Cache (background) → Database
(later)

Advantages

- Very fast writes

Disadvantages

- Risk of losing data before it reaches database
- More complex

Common Usage

- Analytics
- Logging
- Metrics
- IoT
- High-throughput systems

Pattern Comparison

Pattern	Reads	Writes	Complexity	Spring Boot Usage
Cache-Aside	Excellent	Normal	Low	★★★★★ (Most Common)
Read-Through	Excellent	Normal	Medium	Rare
Write-Through	Excellent	Slower	Medium	Rare
Write-Behind	Excellent	Very Fast	High	Specialized

In this course, we use Cache-Aside exclusively.

6. Spring Cache Abstraction

Understanding the Architecture

```
Client → ProductService (@Cacheable) → Spring Cache API → Redis
```

Important: Your service never communicates directly with Redis. It communicates with Spring Cache, which decides which cache implementation to use.

Spring Cache Is an Abstraction

Exactly like:

```
Spring Data JPA → Hibernate  
SLF4J → Logback  
Spring Cache → Redis/Caffeine/EhCache
```

Spring Cache provides:

- Annotations (@Cacheable, @CacheEvict)
- Interfaces
- Configuration

Actual cache implementations provide:

- Storage
- Data management

Request Lifecycle

```
1. GET /products/1  
2. Spring intercepts @Cacheable method  
3. Spring generates cache key: products::1  
4. Spring asks Redis for value  
   └─ Found → Return immediately (cache hit)  
   └─ Not Found → Execute method → Store result → Return (cache miss)
```

Who Does What?

Component	Responsibility
ProductService	Business logic
Spring Cache	Cache abstraction
Redis	Cache storage
PostgreSQL	Persistent storage

Spring Cache Annotations

Annotation	Purpose
@Cacheable	Read from cache or store result
@CacheEvict	Remove cache entries
@CachePut	Update cache without skipping method
@Caching	Combine multiple cache operations
@CacheConfig	Define common cache configuration

Why Not Use RedisTemplate Everywhere?

Using RedisTemplate directly means every service must handle:

- Key generation
- Serialization
- Cache lookup
- Cache insertion
- Cache eviction
- TTL handling

Instead of:

```
@Cacheable("products")
public Product findById(Long id) { ... }
```

You'd write:

```
public Product findById(Long id) {
    String key = "products::" + id;
    Product cached = redisTemplate.opsForValue().get(key);
    if (cached != null) return cached;
    Product product = repository.findById(id);
    redisTemplate.opsForValue().set(key, product, 10, TimeUnit.MINUTES);
    return product;
}
```

For 90% of business applications, Spring Cache is cleaner, safer, and easier to maintain.

When to Use RedisTemplate?

Use RedisTemplate only when you need Redis-specific capabilities:

- Custom data structures
- Counters
- Distributed locks
- Pub/Sub
- Streams
- Lua scripts
- Transactions

7. Implementing @Cacheable

Basic Setup

Step 1 - Add Dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

Why both?

- `spring-boot-starter-cache`: Provides annotations and abstraction
- `spring-boot-starter-data-redis`: Provides Redis client and implementation

Step 2 - Configure Redis

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
```

Step 3 - Configure Spring Cache

```
spring:
  cache:
    type: redis
    redis:
      time-to-live: 5m
      cache-null-values: false
```

Step 4 - Enable Caching

```
@SpringBootApplication
@EnableCaching
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

Without `@EnableCaching`, `@Cacheable` is ignored!

The Simplest Cache

```
@Service
public class ProductService {

    @Cacheable("products")
    public Product findById(Long id) {
        log.info("Loading Product {} from DB", id);
        return repository.findById(id).orElseThrow();
    }
}
```

What Happens Internally?

First Request:

1. GET Product 1
2. Spring Cache checks Redis → MISS
3. Database query executes
4. Result stored in Redis
5. Return result

Second Request:

1. GET Product 1
2. Spring Cache checks Redis → HIT
3. Return cached result
4. Repository is never called

Important Interview Question

Q: Why doesn't the second request execute `repository.findById()`?

A: Spring's cache proxy intercepts the method before the method body executes. If a cached value exists, Spring returns it immediately and skips invoking the method entirely.

Understanding @Cacheable Attributes

Basic Syntax

```
@Cacheable("products")  
// Equivalent to:  
@Cacheable(cacheNames = "products")
```

The Cache Name

Redis stores keys like: `products::1`, `products::2`

The cache name acts like a namespace.

Multiple Caches

```
@Cacheable({"products", "catalog"})
```

Spring checks both caches. If one contains the value, the method is skipped.

Default Cache Key

With `findById(5)`, Spring automatically creates:

- Key = 5
- Redis stores: products::5

Custom Cache Keys

```
@Cacheable(value = "products", key = "#id")
public Product findById(Long id) { ... }
```

Result: products::1 (same as default, but explicit)

What is #id?

This is **Spring Expression Language (SpEL)**. The # symbol means "method parameter."

Multiple Parameters

```
@Cacheable(value = "products", key = "#categoryId + '-' + #productId")
public Product find(Long categoryId, Long productId) { ... }
```

Result: products::10-3 (more readable)

Nested Properties

```
@Cacheable(value = "products", key = "#product.id")
public Product save(Product product) { ... }
```

Result: products::15

Method Calls in Keys

```
@Cacheable(value = "products", key = "#name.toLowerCase()")
public Product findByName(String name) { ... }
```

Result: products::iphone (instead of iPhone)

Static Text in Keys

```
@Cacheable(value = "products", key = "'product-' + #id")
public Product findById(Long id) { ... }
```

Result: products::product-15

Condition - Conditional Caching

```
@Cacheable(value = "products", condition = "#id > 0")
public Product findById(Long id) { ... }
```

- Caches only when ID is positive
- Evaluated **before** method executes
- Uses method parameters only

Unless - Skip Storing Results

```
@Cacheable(value = "products", unless = "#result.price < 100")
public Product findById(Long id) { ... }
```

- Doesn't cache products priced below \$100
- Evaluated **after** method executes
- Can inspect returned object (#result)

Key Difference: condition vs unless

condition	unless
Before method	After method
Uses parameters	Uses returned object
Decides whether caching should start	Decides whether result should be stored

Prevent Cache Stampede with sync

Problem - Cache Stampede: 500 requests arrive simultaneously. None finds data in Redis. All 500 execute `repository.findById()`. Database receives 500 identical queries.

Solution - sync=true:

```
@Cacheable(value = "products", sync = true)
public Product findById(Long id) { ... }
```

- First request loads data
- Other requests wait
- Everyone receives same cached value
- Only one database query executes

When to use sync=true:

- Expensive reports
- Large computations
- Frequently requested data

Not necessary for: Cheap database queries

Complete Example

```
@Cacheable(  
    value = "products",  
    key = "'product-' + #id",  
    condition = "#id > 0",  
    unless = "#result.discontinued",  
    sync = true  
)  
public Product findById(Long id) {  
    return repository.findById(id).orElseThrow();  
}
```

Read it aloud: "Cache Product. Use key 'product-15'. Only for positive IDs. Don't cache discontinued products. Protect against cache stampede."

Summary Table

Attribute	Purpose	Evaluated
value / cacheNames	Cache name	Before
key	Cache key	Before
condition	Cache only if condition is true	Before
unless	Skip storing certain results	After
sync	Prevent cache stampede	During execution
keyGenerator	Custom key generation strategy	Before
cacheManager	Select a specific CacheManager	Before
cacheResolver	Dynamically choose the cache	Before

Best Practices for @Cacheable

1. Use meaningful cache names
2. Keep keys readable

3. Use SpEL instead of manual string building
4. Use `condition` for request-based decisions
5. Use `unless` for response-based decisions
6. Use `sync=true` only when duplicate database work is expensive
7. Avoid caching null values unless explicitly needed

Common Mistakes

1. Using default key when multiple parameters require a clearer key
 2. Building cache keys inconsistently across services
 3. Caching every method without considering read frequency
 4. Forgetting that `unless` is evaluated after method returns
 5. Enabling `sync=true` everywhere (reduces throughput unnecessarily)
-

8. Cache Invalidation Strategies

The Hardest Problem in Caching

Scenario:

- Product is cached: `products::1` → Laptop \$1000
- Someone updates price in database: Laptop \$900
- Redis still contains: Laptop \$1000
- Application is inconsistent
- Some users see \$900, others see \$1000

The Real Problem: Keeping cache synchronized with database is difficult. This process is called **Cache Invalidation**.

Three Strategies

Strategy 1 - Cache Eviction (Recommended)

Delete the cache entry when data changes.

```
@CacheEvict(value = "products", key = "#product.id")
```

```
public Product update(Product product) {  
    return repository.save(product);  
}
```

Flow:

1. Update database
2. Remove from Redis
3. Next request → Cache miss → Database → Fresh cache

Why Eviction is Usually Better

Consider what Product contains:

- Name
- Description
- Images
- Reviews
- Categories
- Price
- Inventory
- Discount
- Rating

Updating Redis correctly means rebuilding the entire cached object. Removing it is much simpler.

Strategy 2 - @CachePut (Update Cache)

Always executes the method, then updates the cache.

```
@CachePut(value = "products", key = "#product.id")  
public Product update(Product product) {  
    return repository.save(product);  
}
```

@CachePut vs @Cacheable

@Cacheable	@CachePut
Skip method on cache hit	Always execute

@Cacheable	@CachePut
Read optimization	Write synchronization

Recommendation: Use `@CacheEvict` for most CRUD services. It's simpler and less error-prone.

Strategy 3 - TTL as Safety Net

Even if a developer forgets cache eviction, Redis automatically removes stale entries.

```
spring:
  cache:
    redis:
      time-to-live: 10m
```

Important: TTL is NOT your primary strategy. Eviction is primary. TTL is backup.

Multiple Cache Entries

Problem: `findById(15)` is cached, AND `findAll()` is cached.

Update Product 15 → Should remove only `products::15`? No, the list cache also became stale.

Solution - @Caching:

```
@Caching(evict = {
    @CacheEvict(value = "products", key = "#product.id"),
    @CacheEvict(value = "products", key = "'all'")
})
public Product update(Product product) {
    return repository.save(product);
}
```

Removing Everything

```
@CacheEvict(value = "products", allEntries = true)
public void clearCache() { }
```

When to use `allEntries=true`:

- Configuration cache
- Small lookup tables

- Reference data

Not suitable for: Large caches with millions of entries.

Before or After Invocation?

```
@CacheEvict(value = "products", key = "#product.id", beforeInvocation = true)
```

- Default: `beforeInvocation = false` (evict after method success)
- With `beforeInvocation = true`: Remove cache before calling method

Recommendation: Default behavior is safer. If database update fails, cache is still valid.

Designing TTL

There is no universal TTL. Choose based on business requirements.

TTL Decision Table

Data	Suggested TTL	Reason
Product Catalog	5–15 min	Mostly read, occasional updates
Categories	1 hour	Rarely changes
Countries	24 hours	Almost static
Currency Rates	5–30 min	Changes periodically
Dashboard Reports	15–60 min	Expensive to compute
User Profile	5 min	Moderate changes
Feature Flags	30 sec–2 min	Operational changes
Inventory Stock	Very short TTL or no cache	Frequently updated
Account Balance	No cache	Strong consistency required

What Should We Cache?

Good Candidates

- Product catalog

- Categories
- Brands
- Configuration
- Countries
- Cities
- Tax rates
- Expensive reports
- Search suggestions
- Frequently requested reference data

What Should NOT Be Cached

Avoid caching data that changes rapidly or requires strong consistency.

Type	Why Not
Inventory stock levels	Changes with every order/return. Stale data allows customers to buy unavailable items
Payment information	Must reflect latest state
Authentication decisions	Roles/permissions may change immediately. Creates security risks
Shopping carts	Highly user-specific, frequently updated
Saga state	Depends on current workflow. Stale data causes incorrect decisions

Note: This doesn't mean Redis is never used in authentication. It's commonly used for session storage, token blacklists, and rate limiting. Just avoid blindly caching authorization results with long-lived `@Cacheable` entries.

Decision Framework

Before adding `@Cacheable`, ask these four questions:

Question 1

Is the data read much more often than it is written?

- YES → Good candidate

- NO → Consider not caching

Question 2

Can the application tolerate slightly stale data?

- YES → Cache is acceptable
- NO → Consider not caching

Question 3

Is retrieving the data expensive?

- YES → Strong candidate
- NO → Consider not caching

Question 4

Will caching significantly reduce database load?

- YES → Cache it
- NO → Consider not caching

If you answer No to most of these questions, don't cache it.

Common Mistakes

1. Forgetting to evict the list cache after updating a single entity
2. Assuming TTL alone guarantees fresh data
3. Using `allEntries=true` for very large caches
4. Choosing arbitrary TTL values ("5 minutes sounds good")
5. Using `@CachePut` when simple eviction is easier and less error-prone

9. Advanced Redis Configuration

Why One TTL Is Usually Wrong

Different business data changes at different rates:

Data	Should TTL Be Same?
Products	10 minutes
Categories	1 hour
Countries	24 hours
Reports	15 minutes
Session data	2 hours
Feature flags	30 seconds

A single TTL for everything rarely makes sense in production.

Per-Cache TTL Configuration

Spring allows configuring different expiration times for different caches.

Custom RedisCacheManager

```
@Configuration
public class CacheConfig {

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory connectionFactory) {

        // Default configuration (applied to caches not explicitly configured)
        RedisCacheConfiguration defaultConfig = RedisCacheConfiguration
            .defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10));

        // Per-cache configurations
        Map<String, RedisCacheConfiguration> cacheConfigurations = new HashMap<>();

        cacheConfigurations.put(
            "products",
            defaultConfig.entryTtl(Duration.ofMinutes(10))
        );

        cacheConfigurations.put(
            "categories",
            defaultConfig.entryTtl(Duration.ofHours(1))
        );

        cacheConfigurations.put(
            "countries",
```

```

        defaultConfig.entryTtl(Duration.ofHours(24))
    );

    cacheConfigurations.put(
        "reports",
        defaultConfig.entryTtl(Duration.ofMinutes(15))
    );

    return RedisCacheManager
        .builder(connectionFactory)
        .cacheDefaults(defaultConfig)
        .withInitialCacheConfigurations(cacheConfigurations)
        .build();
}

```

What Happens?

```

@Cacheable("products") → TTL = 10 minutes
@Cacheable("categories") → TTL = 1 hour
@Cacheable("countries") → TTL = 24 hours
@Cacheable("reports") → TTL = 15 minutes

```

Key Insight: Business logic stays in `@Service`. Infrastructure stays in `@Configuration`. Excellent separation of concerns.

Serialization

Redis stores bytes, not Java objects. Spring must convert Product objects to bytes before sending to Redis.

Option 1 - Java Serialization (Legacy)

Product → ObjectOutputStream → Binary Data → Redis

Problems:

- Not human readable
- Large payload
- Java only
- Version compatibility issues

Option 2 - JSON Serialization (Recommended)

Product → JSON → Redis

Example:

```
{
  "id": 1,
  "name": "Laptop",
  "price": 1200
}
```

Advantages:

- Human readable
- Smaller size
- Language independent
- Easy debugging

Configure JSON Serialization

```
@Bean
public RedisCacheConfiguration redisCacheConfiguration() {
    return RedisCacheConfiguration
        .defaultCacheConfig()
        .serializeValuesWith(
            RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer())
        );
}
```

Why JSON for Microservices

Imagine later you have services in Python, Node.js, and Go reading the same Redis cache.

- Binary Java serialization → Cannot be understood by other languages
- JSON → Can be understood by all languages

Inspecting Redis with redis-cli

```
# Connect to Redis
docker exec -it redis redis-cli

# List all keys
KEYS *
```



```
# Example output
products::1
products::2
```

```
categories::3

# View a value
GET "products::1"

# Check remaining TTL (in seconds)
TTL "products::1"

# Delete a key
DEL "products::1"

# WARNING: Flush all (never use in production!)
FLUSHALL
```

Production Configuration

Development

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
```

Production

Never hardcode. Use:

- **Spring Cloud Config** for configuration
- **Vault** for passwords
- **Different environments** use different Redis clusters

Redis Persistence

Question: "If Redis restarts, is everything lost?"

Answer: Depends on configuration.

Redis supports:

- **RDB snapshots** - Point-in-time snapshots
- **AOF (Append Only File)** - Log of every operation
- **Both together** - Best durability

Should Cache Be Persistent?

Usually **No**.

Remember: Redis cache contains **copies** of database data. If cache disappears, the application simply rebuilds it from the database.

Monitoring Cache Performance

Caching is only useful if it actually reduces database load. Monitor:

- Cache Hits
- Cache Misses
- Hit Ratio
- Evictions
- Memory Usage
- Latency

These metrics are available through:

- Spring Boot Actuator
- Micrometer
- Prometheus
- Grafana

Common Production Mistakes

1. ☐ One TTL for every cache
2. ☐ Java serialization in modern distributed systems
3. ☐ Using `KEYS *` on production systems (blocks Redis; use `SCAN` instead)
4. ☐ Forgetting memory limits and eviction policies
5. ☐ Treating Redis as the primary database
6. ☐ Hardcoding Redis credentials

Best Practices

1. Separate TTL by business domain

2. Prefer JSON serialization for interoperability
 3. Keep business logic free from cache configuration
 4. Use meaningful cache names
 5. Verify cache contents using redis-cli during development
 6. Externalize Redis configuration
 7. Monitor cache hit ratio
-

10. Live Coding Workshop

This workshop builds caching step-by-step while observing what happens.

Phase 1 - Verify Redis is Running

```
docker ps
```

Phase 2 - Enable Spring Cache

Before:

```
@SpringBootApplication
public class ProductServiceApplication {
    // No caching
}
```

After:

```
@SpringBootApplication
@EnableCaching
public class ProductServiceApplication {
    // Caching enabled
}
```

Observe: No cache → Cache enabled

Phase 3 - Add @Cacheable

```
@Service
public class ProductService {

    @Cacheable("products")
```



```
    public Product findById(Long id) {  
        log.info("Loading Product {} from DB", id);  
        return repository.findById(id).orElseThrow();  
    }  
}
```

Test:

```
# First request  
curl http://localhost:8080/products/1  
# Log: Loading Product 1 from DB  
  
# Second request  
curl http://localhost:8080/products/1  
# Log: (No DB log)
```

Observe:

- First request: CACHE MISS
- Second request: CACHE HIT

Phase 4 - Inspect Redis

```
# List all keys  
docker exec -it redis redis-cli KEYS "*"   
  
# View a value  
docker exec -it redis redis-cli GET "products::1"  
  
# Check TTL  
docker exec -it redis redis-cli TTL "products::1"
```

Phase 5 - Update Product

```
@CacheEvict(value = "products", key = "#product.id")  
public Product update(Product product) {  
    return repository.save(product);  
}
```

Observe:

1. Cache removed
2. Database updated
3. Next request reloads cache

Phase 6 - Change TTL

```
spring:
  cache:
    redis:
      time-to-live: 10s # Very short TTL
```

Observe:

1. Request → Cache miss → Load from DB
2. Wait 10 seconds
3. Request again → Cache miss again

Phase 7 - Performance Comparison

Without Cache:

```
Request 1: 150ms (DB query)
Request 2: 145ms (DB query)
Request 3: 148ms (DB query)
```

With Cache:

```
Request 1: 150ms (DB query)
Request 2: 5ms (Cache hit)
Request 3: 4ms (Cache hit)
```

Phase 8 - Common Mistakes

Mistake 1: Missing @EnableCaching

```
// Remove @EnableCaching
@SpringBootApplication
public class ProductServiceApplication {
    // No @EnableCaching
}
```

Result: Nothing is cached. `@Cacheable` is ignored.

Mistake 2: Missing @CacheEvict

```
// Update method without @CacheEvict
public Product update(Product product) {
    return repository.save(product);
}
```

Result: Stale data remains in cache.

Mistake 3: Wrong Key

```
@Cacheable(value = "products", key = "#productId") // Wrong parameter name
public Product findById(Long id) {
    // ...
}
```

Result: Duplicate cache entries or cache misses.

11. Common Pitfalls and Solutions

Pitfall 1: Cache Stampede

Problem: Multiple concurrent requests miss cache and hit database simultaneously.

Solution:

```
@Cacheable(value = "products", sync = true)
public Product findById(Long id) { ... }
```

Pitfall 2: Cache Avalanche

Problem: All keys expire at the same time, causing database overload.

Solution: Randomize TTL.

```
// Random TTL between 8-12 minutes
Duration.ofMinutes(10).plusSeconds(new Random().nextInt(120))
```

Pitfall 3: Cache Penetration

Problem: Requests for non-existent data (e.g., Product 999999) hit database every time.

Solutions:

- Short-lived negative caching (when appropriate)
- Bloom Filter (advanced systems)
- Rate limiting

Pitfall 4: Cache Breakdown (Hot Key)

Problem: One extremely popular product expires, causing millions of requests to hit the database.

Solutions:

- `sync=true`
- Background refresh
- Longer TTL for hot keys

Pitfall 5: Redis Unavailable

Problem: What happens when Redis goes down?

Solution: Degrade gracefully.

```
Redis Down → Go to Database → Application still works
```

Implementation: Configure timeout and fallback.

Pitfall 6: Inconsistent Caches

Problem: Multiple service instances have different cached values.

Solution: Use Redis (distributed cache) instead of local cache.

Pitfall 7: Memory Exhaustion

Problem: Unlimited cache growth consumes all memory.

Solution: Set maximum size and eviction policies.

```
Caffeine.newBuilder()  
    .maximumSize(1000) // Limit to 1000 entries  
    .build();
```

Pitfall 8: Too Much Caching

Problem: Caching data that doesn't benefit from caching.

Solution: Use the decision framework:

1. Is data read more than written?
 2. Can application tolerate stale data?
 3. Is retrieval expensive?
 4. Will caching reduce database load?
-

12. Quick Reference

Spring Cache Annotations

Annotation	Purpose	Example
@Cacheable	Read from cache or store result	@Cacheable("products")
@CacheEvict	Remove cache entries	@CacheEvict(value="products", key="#id")
@CachePut	Update cache without skipping method	@CachePut(value="products", key="#product.id")
@Caching	Combine multiple operations	@Caching(evict = {...})
@CacheConfig	Shared configuration	@CacheConfig(cacheNames = "products")

Common SpEL Expressions

Expression	Meaning
#id	Method parameter named 'id'
#product.id	Property of product parameter
#result	Method return value
#name.toLowerCase()	Method call
'prefix-' + #id	String concatenation

TTL Recommendations

Data Type	TTL
Products	10 min
Categories	1 hour
Countries	24 hours
Reports	15–60 min
Inventory	No cache
Orders	Usually no cache
User sessions	2 hours

Configuration Checklist

1. Dependencies

```
spring-boot-starter-cache  
spring-boot-starter-data-redis
```

2. Configuration

```
spring:  
  cache:  
    type: redis  
  data:  
    redis:  
      host: localhost  
      port: 6379
```

3. Enable Caching

```
@EnableCaching
```

4. Add Annotations

```
@Cacheable(value = "products", key = "#id", sync = true)
```

5. Evict on Updates

```
@CacheEvict(value = "products", key = "#product.id")
```

Decision Matrix

Requirement	Recommendation
Cache method results	@Cacheable
Remove cache after update	@CacheEvict
Always refresh cache	@CachePut
Use Redis data structures	RedisTemplate
Distributed locks	RedisTemplate
Pub/Sub	RedisTemplate
Redis Streams	RedisTemplate

Interview Questions

Q1: What is the difference between condition and unless?

A: `condition` is evaluated before method execution and uses method parameters. `unless` is evaluated after method execution and can inspect the returned object (`#result`) to decide whether it should be cached.

Q2: Why would you use `sync=true`?

A: To prevent a cache stampede. When multiple concurrent requests miss the cache for the same key, only one request loads the data while others wait for the cached result.

Q3: Why use a custom cache key?

A: To create readable, consistent, and collision-free keys, especially for methods with multiple parameters or when sharing cache conventions across services.

Q4: Why is cache invalidation considered difficult?

A: Once data is cached, every database change must be reflected in the cache at the correct time. Missing an invalidation causes stale data, while unnecessary invalidation reduces cache effectiveness.

Q5: When would you choose @CachePut instead of @CacheEvict?

A: When the updated object returned by the method is exactly what should be stored in the cache, and keeping the cache warm is more beneficial than forcing the next request to reload from the database.

Q6: Why is TTL considered a safety net?

A: TTL eventually removes stale entries even if a write path forgot to evict them. However, it cannot guarantee immediate consistency after updates, so it should complement—not replace—explicit cache eviction.

Q7: Why shouldn't every cache have the same TTL?

A: Different business data changes at different frequencies. Product catalogs, countries, reports, and inventory each have different freshness requirements.

Q8: Why is JSON preferred over Java serialization?

A: JSON is human-readable, has smaller payload, is language independent, and easier to debug. Java serialization is not readable, has larger payload, and only works with Java.

Architecture Review



Final Checklist

Before deploying caching to production:

- ☐ Verified caching reduces database load
- ☐ Configured appropriate TTL per cache
- ☐ Implemented cache eviction on updates
- ☐ Set maximum cache size
- ☐ Configured serialization (JSON preferred)
- ☐ Monitored cache hit ratio
- ☐ Tested Redis failure scenarios
- ☐ Documented cache configuration
- ☐ Externalized Redis connection details
- ☐ Set up monitoring and alerts

End of Tutorial